Satvik: handbook

Linking, tree models and metrics

Read 00_Start_Here first; then this handbook.

Machine Learning (702DB0C015), B.Tech CSE (Data Science), Semester V, 2026-27

Contents

1. Part brief
2. Knowledge pack
3. Presentation guide
4. Viva questions and answers
5. Diagrams

Satvik: linking, the tracklet classifier (trees) and metrics

> **Update: classical ML only (bundle v6).** Stage 1 is now XGBoost on 29 stamp features + PCA instead of the CNN (`project/docs/classical_realbogus.md`). Your part is unchanged: you consume its `score` column and threshold (v6: 0.0052, 99% recall on validation); the tracklet model was retrained on those scores (v6 tracklet cut 0.533). Where this folder says "CNN score", read "stage-1 score".

Marks this part carries: Model Development (4, shared with Paras) and the evaluation-metrics foundation of Model Evaluation (3, shared with Advik). **Why it matters:** this is the classical-ML half of the project (logistic regression, trees, forests, boosting, cross-validation) and the metric thinking that every model in the project is judged by.

Your four jobs

1. **Linking:** how blobs from different frames become straight-line tracks (a search, not learning), why it is fast, and the biggest bug of the project (pointing shifts).
2. **The tracklet classifier:** the 17 features and the physical idea behind each, the logistic regression baseline, decision tree -> random forest -> gradient boosting, cross-validation by night, the operating point.
3. **Metrics and evaluation basics:** confusion matrix, accuracy paradox, precision/recall/FPR, average precision, PR vs ROC, choosing a threshold on validation, calibration and base-rate adjustment.
4. **Cross-match and vetting:** separating known from unknown asteroids, the expected number of false survivors, and the checklist for a human reviewing a candidate.

What is NOT yours

Downloads and detection: Aaryan. Labels, stamps, the CNN: Paras (you consume its `score` column and threshold). Scoreboard, deep search, viewer: Advik.

Where your code is (in `project/`)

Area	Files	What they do
Linking	`src/asthunt/link/fit_linear_motion.py` , `link_detections_into_tracklets.py` , `measure_sky_motion.py` , `link_tracklets_across_nights.py`	straight-line fit; the seed-and-predict linker with KD-trees; speed on the sky; (two-night linker, brief)

Area	Files	What they do
Features	`src/asthunt/features/compute_tracklet_features.py`, `count_static_neighbours.py`, `measure_forced_photometry.py`, `add_path_features.py`	the 11 + 2 + 4 = 17 numbers per tracklet
Models	`src/asthunt/models/train_tracklet_classifier.py`, `train_baseline_classifier.py`	XGBoost; logistic-regression baseline
Metrics	`src/asthunt/evaluate/compute_classification_metrics.py`, `choose_threshold.py`, `calibrate_probabilities.py`, `adjust_for_base_rate.py`	the numbers and the thresholds
Labels and cross-match	`src/asthunt/label/label_tracklets.py`, `src/asthunt/crossmatch/crossmatch_known_asteroids.py`, `src/asthunt/report/write_candidate_report.py`, `write_ades_report.py`	tracklet labels; known vs unknown; the HTML report and draft ADES file
Config	`configs/link.yaml`	linking limits
Tests	`tests/link`, `tests/features`, `tests/evaluate`, `tests/crossmatch`, `tests/report`	
Lessons and notebooks	`docs/ml_basics.md`, `linking.md`, `tracklet_classifier.md`, `metrics_evaluation.md`, `crossmatch_vetting.md`, `multi_night_linking.md`; notebooks with the same names	

What you hand over

- The classical-ML section and the metrics definitions for the report.
- Figures: the linking picture, the model comparison table, the confusion matrix and the PR curve (from the metrics notebook).
- The "how a human vets a candidate" checklist.

Your seven key messages

1. **Linking is a search, not learning:** physics says an asteroid moves in a straight line at constant speed; a KD-tree makes the search fast.
2. **We never try all combinations** (about 2 x 10^19); a pair of blobs defines a line and we look up the rest, after the CNN has removed about 76% of blobs.
3. **The telescope pointing shifts between exposures**, so linking must use a common sky grid; ignoring that was the biggest bug of the project.
4. **Every feature has a physical reason** (steady speed, steady brightness, shape, what the CNN thought, what the sky shows at the predicted spot).
5. **Trees fit tables of numbers best;** four models tied in cross-validation by night, so the choice is practical, and that honesty is a result.
6. **Accuracy is the wrong metric:** "always bogus" scores 96.8%; use recall, precision, average precision, and fix recall first.

7. **A human makes the final call,** with a written checklist; and we always look at the pictures, not only numbers.

Numbers you must know by heart

Number	Meaning	Source
about 2 x 10^19	combinations if every blob were tried in every frame	`docs/linking.md`
52,000 -> 3,500	seed pairs before/after the CNN filter (about 15x cheaper); 0.2 s per night	same
8 arcsec/hour	minimum fitted speed (centroid noise fakes slower speeds)	`configs/link.yaml`
3 of 5 frames; 1.5 px; 2 px	minimum detections (v4; v1 used 4 of 5), max residual, match radius	`configs/link.yaml`, bundle
2.5 px -> 0.05-0.47 px	real asteroids' straight-line residual before/after the common sky grid	`docs/linking.md`
11 + 2 + 4 = 17	features per tracklet	`features/`
300 trees, depth 3, lr 0.05, 80% / 80%	XGBoost settings	`train_tracklet_classifier.py`
0.999 / 0.983 / 0.999 / 0.999	cross-validated AP: logistic regression, tree, forest, XGBoost	`docs/tracklet_classifier.md`
96% real tracklets	why tracklet AP is saturated (we plant 300 fakes per night)	same
96.8% vs 98.4%	accuracy of "always bogus" vs the baseline	`docs/metrics_evaluation.md`
0.95 vs 0.78	ROC AUC vs average precision of the baseline	same
1,547 / 582 / 493 / 64,419	TP / FN / FP / TN of the baseline at threshold 0.5	same
0.082; 19.3%; 804	threshold, precision and false positives per frame at 90% recall (baseline)	same
9 of 9	"false positives" on the test night that were real known asteroids	`docs/tracklet_classifier.md`

Hands-on tasks (do these yourself)

1. `cd project && pytest tests/link tests/features tests/evaluate tests/crossmatch -q`.
2. Re-run `notebooks/ml_basics.ipynb`, `metrics_evaluation.ipynb` and `tracklet_classifier.ipynb`. Reproduce the confusion matrix and the four-model comparison.
3. Compute by hand: with 2,099 real and 64,974 bogus, FPR 2% and recall 95%, what is the precision? (answer in `04_Viva_QA.md`).
4. In `configs/link.yaml` set `min_rate_arcsec_per_hour` to 3 in a copy and predict how many more "tracklets" appear and why most would be junk.
5. Train a depth-3 decision tree on the tracklet features and read out its first three questions in words.

Pitfalls

- Reporting accuracy or ROC AUC alone for a rare-class problem.
- Using random `KFold` on grouped data (inflated scores): always group by night.
- Trusting AP from a handful of negatives (38 bogus tracklets in training, 9 in validation).
- Calling a tracklet "bogus" in a label table without checking: all 9 test-night "false positives" were real known asteroids.
- Quoting balanced-training probabilities as real-world probabilities.

Satvik: knowledge pack

0. Read first (in this order)

`00_Start_Here/01_Project_Overview` -> `02_Domain_Primer` (sections 8-10) -> `03_ML_Primer` (sections B, C, D) -> then `project/docs/ml_basics.md`, `linking.md`, `tracklet_classifier.md`, `metrics_evaluation.md`, `crossmatch_vetting.md`.

---

1. Linking: from blobs to tracks

Goal: join detections in the 5 frames into **tracklets**: sets of detections on one straight line at a plausible speed. It is a *search*, not a learning step. Physics supplies the rule: over about 30 minutes an asteroid moves in a straight line at near-constant speed.

1.1 Never try every combination

5 frames x about 8,000 blobs: about 2 x 10^19 combinations. Instead (see `07_Diagrams/01_linking.svg`):

1. **Seed:** for each pair of frames, take every pair of detections whose implied speed is between the minimum and maximum. A KD-tree `query_ball_point` finds all neighbours within `max_rate x dt` instantly.
2. **Predict:** a seed fixes a velocity, so we know where the object must be in every other frame; a KD-tree `query` finds the nearest detection there (within 2 px).
3. **Keep:** at least 3 of 5 frames (v4; v1 required 4 of 5), straight-line residual at most 1.5 px, **fitted** speed within limits.
4. **De-duplicate:** many seeds find the same tracklet; take them best-first (most detections, smallest residual) and drop any that re-use a detection. **Using the CNN to shrink the search:** keeping 95% of real detections (a threshold chosen on validation) removes about 76% of detections; seed pairs fall from 52,000 to 3,500 (about 15x cheaper); linking a night takes about 0.2 s. **Why fits and not only seeds:** the speed limit must be checked on the *fitted* track, not just on seed pairs (a bug we had and fixed with a test).

1.2 The biggest bug: the telescope moves between exposures

On the test night bright known asteroids failed to link while fakes linked perfectly (100%). Every real asteroid had the **same** residual pattern (0.6, 2.5, 2.4, 0.6, 1.7 px): identical residuals across different objects point to a **per-frame** error, not to orbits.

- **Cause:** pointing shifts between exposures (up to 12 px on the test night, 4-12 px on others), and we linked in each frame's own pixels.
- **Why it hid:** fakes were planted moving straight *in pixel coordinates*, so they ignored the shifts. On most nights the drift was smooth and looked like extra velocity; on the test night it jumped.

- **Fix:** every detection gets `x_common, y_common` (its RA/Dec projected onto frame 1's pixel grid) and linking uses those; fakes are planned on frame 1's grid and converted through the sky into each frame's pixels.
- **Check:** real asteroids' residuals fell from about 2.5 px to **0.05-0.47 px**.

1.3 The minimum speed must beat measurement noise

After the fix the test night still had 78 "unknown" tracklets at a median 2.5 arcsec/hour: stationary sources (bad subtractions, variable stars) whose centroids wobble. With about 0.5 px centroid noise over 28 minutes the fitted speed noise is about 1.4 px/hour per axis, so a stationary source can "move" at 3-6 arcsec/hour. New minimum: **8 arcsec/hour**, still far below the speeds of the asteroids we recover (28-48). Cost: fakes planted slower than 8 (18 of 300 per night) cannot be linked, by design.

1.4 Results

Linking efficiency (linked among linkable fakes) **100%** on the training night shown; tracklets per night 176-212 on training/validation nights (mostly our 300 fakes) with only **1-8 bogus**; on the test night 273 tracklets (34 labelled bogus, of which 14 were known asteroids our labels missed). Bogus ones are **near misses** (a fake plus a noise detection) or chance alignments: valuable hard negatives.

1.5 Two-night linking (brief)

Tracklets from different nights are joined with a tolerance that grows with the distance travelled; 5 of 5 known asteroids were linked over 3 days with 0 wrong. A candidate means little until it is seen on a second night (the Minor Planet Center needs two).

---

2. The tracklet classifier

2.1 The 17 features and the idea behind each

group	features	idea
motion	`n_detections`, `rate_arcsec_per_hour`, `max_residual`	steady, straight motion
brightness	`mag_mean`, `mag_std`	an asteroid keeps nearly the same brightness for 30 minutes; noise blips vary
strength	`snr_mean`, `snr_min`	one weak link makes a weak track
CNN opinion	`score_mean`, `score_min`	what the CNN thought of each detection (cross-fitted for training nights)
shape	`npix_std`, `elongation_mean`	a point source keeps the same size and roundness

group	features	idea
static neighbours (v4)	`static_neighbours_mean`, `_max`	detections left behind at the track's own positions in other frames: a mover leaves none, an artifact walker leaves many
forced photometry (v4)	`forced_snr_min`, `forced_snr_median`, `forced_snr_std`, `stack_snr`	the SNR at the predicted position in *every* frame, detected or not; real objects leave a weak blob even in frames where they were missed; noise alignments leave nothing
Why the last two groups exist: vetting 15 false alarms of v2 by eye showed two failure modes (artifact walkers, noise alignments); each got one feature. **A bug in my own feature:** it counted a slow asteroid's own detections as "static neighbours", flagging 22% of real tracklets; fixed to 0.4% with a test that reproduces it.		

2.2 Decision tree -> random forest -> gradient boosting (`07_Diagrams/02_boosting.svg`)

- **Decision tree:** repeated yes/no splits that best separate the classes; readable, but one tree over- or under-fits.
- **Random forest (bagging):** hundreds of trees, each on a random sample of rows and features; average their votes; lower variance.
- **Gradient boosting (XGBoost):** trees built one after another, each fitting the *errors* of the trees so far, scaled by a small learning rate. Usually the most accurate on tables.
- **Our settings** (deliberately modest: about 1,600 tracklets): 300 trees, depth 3, learning rate 0.05, each tree sees 80% of rows and 80% of columns.
- **Why trees for this table:** features on different scales, no scaling needed, handles missing values and interactions.

2.3 Cross-validated by night (`GroupKFold`; average precision per fold)

model	folds	mean
logistic regression	0.999 / 0.999 / 0.999 / 0.999	0.999
decision tree (depth 3)	0.987 / 0.979 / 0.992 / 0.974	0.983

model	folds	mean
random forest	0.999 / 0.999 / 1.000 / 0.999	0.999
XGBoost	1.000 / 0.999 / 1.000 / 0.999	0.999
Everything except the single tree is near-perfect and the folds overlap: **a statistical tie.** XGBoost is used because it handles missing values and interactions without extra work, but logistic regression would be a defensible, simpler choice. Fancier is not automatically better.		

2.4 Why it looks so easy, and what really matters

In this dataset **96% of tracklets are real** (300 planted fakes per night, junk mostly removed by the CNN and linker): the opposite of the real sky. So AP about 0.999 only says the features separate *these* examples. The useful numbers: **bogus tracklets per night above the threshold** (about 2 per night from 9 validation bogus examples, "a few", not "exactly 2"), depth, known asteroids recovered.

2.5 Operating point

Keep 95% of real tracklets on validation. In the first lesson that gave a cut of 0.980 (precision 0.989); the live bundle v4 was retrained on harder tracklets with 17 features and uses **0.601**. Recall by brightness (validation, v1): at least 98% brighter than 19.5, 93% at 19.5-20, **79%** at 20-20.5: the faint end is where it loses.

2.6 The test night (looked at once)

AP 0.988, recall 0.887, 9 tracklets labelled "bogus" were kept: **all 9 were real known asteroids** (V 17.5-21.2) that labels missed (fainter than the V <= 20.5 labelling cut, or a frame where the match failed). True false positives on the test night: **0**; the labels were wrong.

---

3. Metrics and evaluation basics (`07_Diagrams/04_confusion_metrics.svg`)

3.1 The baseline (ML basics): logistic regression on 6 numbers

`p = sigmoid(w.x + b)` , trained by minimising log-loss; features `snr, peak, npix, a, b, flux` ; `StandardScaler` inside a pipeline (fitted on training data only); stratified split with a fixed seed; L2 regularisation (strength `C`). On the toy tracklet data: 99.8% (confusion 440 / 2 / 0 / 441); log10 of rate and residual gave 100%, which meant the data was too easy (see Aaryan's leakage audit).

3.2 Confusion matrix on 67,041 validation detections (2,129 real)

`TP = 1,547, FN = 582, FP = 493, TN = 64,419` at threshold 0.5. Recall = 1,547 / 2,129 = 72.7%; precision = 1,547 / 2,040 = 75.8%; FPR = 493 / 64,912 = 0.76%; 493 false positives over 10 frames =

about 49 per frame.

3.3 Accuracy fails

"Always bogus" gets **96.8%** accuracy and 0% recall; the baseline gets 98.4% with 72.7% recall. Almost the same accuracy, completely different usefulness.

3.4 Precision-recall vs ROC

ROC AUC 0.95 looked excellent; average precision was 0.78. ROC's x-axis divides by all bogus (65,000), so hundreds of false alarms look like a tiny step; precision divides by the number of alerts. **For rare positives use precision-recall.**

3.5 Choosing the operating point

A missed asteroid is a lost discovery; a false alarm costs a person a minute. So fix a recall target (90%) and minimise false alarms. `choose_threshold(labels, scores, 0.90)` gave 0.082: recall 90.0%, precision 19.3%, **804 false positives per frame** (the CNN brings this to 176). Pushing from 73% to 90% recall multiplied false alarms by 16. **Always choose thresholds on validation, never on test.**

3.6 Calibration: do scores mean probabilities?

Training was balanced (16.7% real) but the truth is 3.1%, so raw scores are inflated (a prediction of 0.6 was real only about 30% of the time). Two fixes: **base-rate adjustment** (Bayes in odds form: `new odds = model odds x (true prior odds / training prior odds)`; example from the docstring: 0.9 becomes 0.043 when real is 1 in 1,000 instead of 1 in 6) and **isotonic regression** (a never-decreasing curve from scores to observed fractions), fitted on one half of validation and checked on the other half (fitting and judging on the same rows would flatter it). Calibration changes what you can *say*, not the ranking.

3.7 Completeness curves

Completeness by brightness with error bars `sqrt(p(1−p)/n)`: with p = 0.7 and n = 30 the uncertainty is +/- 0.084, so bins of 30 objects mostly show noise.

---

4. Cross-match and vetting

- **Cross-match:** for every detection find the nearest catalogue prediction in its frame (4 px, covering catalogue offsets); a tracklet is "known" if at least half its detections match the *same* object, so chance coincidences are negligible.
- **Real search on validation night 2023-09-15:** 31,953 detections -> 13 tracklets -> 10 known asteroids, 0 unknown candidates; of 7 known asteroids with V <= 20 present in at least 4 frames, 5 recovered.
- **Look-elsewhere effect:** thousands of seed combinations per night means some junk survives every cut; estimate the expected number before looking at candidates (about 2 per night): if a night yields about that many unknown candidates, most are probably bogus.

- **Vetting checklist** for every unknown candidate: (1) stamps: point source in science and difference, nothing in reference, every frame? (2) motion steady and plausible (main-belt near opposition: 25-50 arcsec/hour roughly westward)? (3) brightness roughly constant? (4) not on a galaxy or bright star, not along a bleed trail, not a satellite streak? (5) no catalogue match even at a looser radius (15 px)? (6) seen on another night (the MPC needs at least two)?
- **Lesson from the test night:** three interesting unknowns looked like main-belt asteroids in the numbers but were a galaxy residual, satellite trails and a saturated-star bleed in the pictures. **Always look at the stamps.**
- **ADES draft:** the format for reporting to the Minor Planet Center; ours uses mid-exposure times (shutter-open + 15 s), a temporary designation and a header saying "DRAFT ... NOT FOR SUBMISSION".

---

5. Cross-part cheat sheet

- **Aaryan:** ZTF difference images; unit = field+CCD+quadrant+night; split by night (8/2/1/1); noise via MAD; blobs by SEP at 2 sigma; the leakage audit.
- **Paras:** labels (planted fakes, Horizons, "the rest"); 3x31x31 stamps; CNN with 72,145 weights; AP 0.899 vs 0.780; cross-fitting by night; the label bug found by the model.
- **Advik:** one deployed path; scoreboard on planted fakes and held-out nights; depth 20.5 -> 21.4 -> 22.1; deep search with a 6-sigma cut; live viewer; a multi-frame CNN tied XGBoost.

6. Self-quiz (answers in `04_Viva_QA.md`)

1. Why do identical residual patterns across different asteroids point to a per-frame problem? 2. How does the CNN filter make linking about 15x cheaper? 3. Centroid noise 0.5 px over 30 minutes: what false speed can a stationary source show? 4. What does each new tree in boosting try to fix? 5. Four models score 0.98-0.999 with overlapping folds: how do you choose? 6. Why is "bogus tracklets per night" more useful than AP? 7. What did you check before believing 9 false positives on the test night? 8. Why do we fix recall first? 9. 2,099 real, 64,974 bogus, FPR 2%, recall 95%: precision? 10. A model trained on 50% real says 0.8; real is 3%: adjusted probability?

Satvik: presentation guide

Your slot: 5 minutes, 5 slides. (In the 12-minute plan: 3 minutes, slides 1, 3 and 4.) Pictures are in `07_Diagrams/`.

Slide 1: "From blobs to tracks" (1 min)

On the slide: `linking.svg`. **Say:** "The network gives us thousands of plausible blobs per frame. An asteroid, though, moves in a nearly straight line at a steady speed over the half hour of five exposures. So we join blobs from different frames that line up that way. This is a search, not learning. Trying every combination would be about two times ten to the nineteen tracks, so instead two blobs in two frames define a line, and we look up where the object must be in the other frames with a KD-tree. Because the network has already removed about three quarters of the blobs, a night takes about a fifth of a second. The biggest bug of the whole project lived here: the telescope does not point at exactly the same pixels each exposure, up to twelve pixels of shift, so we first put every blob on one common sky grid. Real asteroids' straight-line error dropped from about two and a half pixels to under half a pixel."

Slide 2: "Seventeen numbers per track" (1 min)

On the slide: the feature table from your knowledge pack, grouped: motion, brightness, strength, CNN opinion, shape, static neighbours, forced photometry. **Say:** "Each track becomes seventeen numbers, and each has a physical reason: steady speed and a straight line; steady brightness, because noise blips vary; the weakest link; what the neural network thought; a steady shape. Two groups were added after we looked at false alarms by eye: 'static neighbours', to catch artifacts that sit in the same place in every frame, and 'forced photometry', which measures the brightness at the predicted spot even in frames where nothing was detected, because a real object leaves a weak blob there and noise does not. I should also say that I found a bug in my own feature: it counted a slow asteroid's own detections as static neighbours and wrongly flagged 22% of real tracks; I caught it by checking on real examples, fixed it and wrote a test."

Slide 3: "Trees: from one to boosting" (1 min)

On the slide: `boosting.svg`. **Say:** "Tables of numbers on different scales are where tree models work best. A decision tree asks a few yes or no questions; a random forest averages hundreds of random trees; gradient boosting builds trees one after another, each correcting the errors of the ones before. We used XGBoost with deliberately modest settings: 300 shallow trees, small learning rate, each tree seeing 80% of rows and columns, because we have only about 1,600 tracklets. We cross-validated by night, never by random rows. The honest result: logistic regression, forest and XGBoost all scored about 0.999 and their folds overlap: a tie. We kept XGBoost because it handles missing values and interactions without extra work, but a simpler model would also be defensible."

Slide 4: "Why we do not report accuracy" (1 min)

On the slide: `confusion_metrics.svg`. **Say:** "Only about 3% of blobs are real. A model that says 'bogus' every time is 96.8% accurate and finds nothing. So we use the confusion matrix: of 2,129 real objects the baseline found 1,547, recall 73%; of the 2,040 things it flagged, 76% were real, precision. We prefer

precision-recall to ROC for rare positives: ROC AUC looked excellent at 0.95 while precision at a useful recall was under 20%. And we choose the threshold on validation nights, never on test."

Slide 5: "Operating point, cross-match, and the human" (1 min)

On the slide: `operating_point.svg` plus three bullets: fix recall; compare with the catalogue; a person looks. **Say:** "A missed asteroid is a lost discovery; a false alarm costs a person a minute. So we fix recall at 90% on validation and then count false alarms: 804 per frame for the simple baseline, 176 for the network. Each surviving track is compared with the catalogue of known asteroids; nearly everything is known. For anything unknown there is a written checklist: does it look like a point source in science and difference but not in the reference, is the motion plausible, is it on a galaxy or a bleed trail, is there any catalogue match, was it seen on another night? On our test night the numbers made three unknown tracks look like asteroids, but the pictures showed a galaxy residual, satellite trails and a bleed trail. We always look at the pictures. Over to Advik, who will show how we measure all of this honestly."

Delivery tips

- Slide 4 is your strongest: say the 96.8% line slowly.
- Use "recall" and "precision" with one plain sentence each the first time.
- For numbers, round and say what they are: "about 800 false alarms per frame", not "804.2".

Anticipated first questions

1. "Why XGBoost and not a neural network?" -> small table of 17 numbers, about 1,600 examples: trees are strong, need no scaling; networks would overfit. 2. "Why did models tie?" -> the features carry the signal; simplicity. 3. "How do you pick the threshold?" -> recall target on validation. 4. "What is the baseline?" -> logistic regression on six numbers. 5. "Why cross-validate by night?" -> same-night leakage.

Satvik: viva questions and model answers

A. Linking

1. What is a tracklet and how is it found? A set of detections in different frames on one straight line at a plausible speed. Seed: a pair of detections in two frames at a plausible speed; predict: that fixes a line, so look up the nearest detection where the object must be in the other frames (KD-tree, 2 px); keep: at least 3 of 5 frames, within 1.5 px of the fitted line, fitted speed 8-120 arcsec/hour; de-duplicate best-first.

2. Why not try every combination? 5 frames x about 8,000 blobs gives about 2 x 10^19 combinations. A pair of blobs defines a line, so we look the rest up instead of enumerating.

3. How does the CNN make linking cheaper? Keeping 95% of real detections removes about 76% of blobs; seed pairs fall from 52,000 to 3,500 (about 15x) and a night links in about 0.2 s.

4. Why do identical residual patterns across different asteroids point to a per-frame problem? Different orbits would give different residuals. The same pattern (0.6, 2.5, 2.4, 0.6, 1.7 px) for every asteroid means something common to each frame: the pointing shift.

5. Explain the common sky grid. Every blob's RA/Dec is projected onto frame 1's pixel grid (`x_common`, `y_common`), because asteroids move in straight lines on the sky, not in each frame's pixels. Residuals dropped from about 2.5 px to 0.05-0.47 px.

6. Why was the bug hidden? Fakes were planted moving straight in pixel coordinates, so they ignored the pointing shifts and linked perfectly. Our simulation shared the pipeline's blind spot. Lesson: test the simulation against real objects.

7. Why is the minimum speed 8 arcsec/hour? Centroid noise (about 0.5 px over 28 minutes) makes a stationary source appear to move at 3-6 arcsec/hour; 78 such tracklets appeared on the test night. 8 is still far below the 28-48 arcsec/hour of the asteroids we recover. Cost: fakes slower than 8 (18 of 300) cannot be linked, by design.

8. Centroid noise is 0.5 px over 30 minutes. Roughly what false speed can a stationary source show? Fitted speed noise is about 1.4 px/hour per axis (px/hour is about arcsec/hour here), so false speeds of several arcsec/hour appear; hence a minimum well above that.

9. Why check the speed on the fitted track and not only on seeds? A seed pair can pass the limit while the final fitted track does not; the check on seeds alone let slow tracks through. Fixed, with a test.

B. Features and models

10. What features describe a tracklet and why? 17 numbers: motion (count, rate, max residual), brightness (mean, std), strength (mean and min SNR), CNN opinion (mean and min score), shape (size spread, elongation), static neighbours (mean, max), forced photometry (min, median, std of the SNR at the predicted spot, stack SNR). Each reflects physical behaviour: a mover is steady, round, leaves nothing behind and leaves a weak blob where it was missed.

11. What are static neighbours and forced photometry? Static neighbours count detections left at the track's own positions in other frames: a mover leaves none, an artifact walker leaves many. Forced photometry measures SNR at the predicted position in every frame, detected or not; real objects leave a weak blob, noise alignments leave nothing.

12. What bug did your static-neighbour feature have? It counted a slow asteroid's own detections as neighbours and flagged 22% of real tracklets. Found by checking the feature on real examples; fixed to 0.4%; a test reproduces it; v3 (trained with the bug) was retired and v4 trained correctly.

13. Why trees for this problem? A small table of numbers on different scales, about 1,600 examples: trees need no scaling, handle interactions and missing values, and are the standard strong choice.

14. Explain decision tree, random forest and gradient boosting. Tree: yes/no splits; forest: many random trees averaged (bagging, lower variance); boosting: trees built sequentially, each fitting the errors of the previous ones with a small learning rate.

15. What are your XGBoost settings and why? 300 trees, depth 3, learning rate 0.05, subsample 0.8, colsample 0.8: modest on purpose for about 1,600 tracklets; shallow trees and subsampling limit overfitting.

16. Four models scored 0.98-0.999 with overlapping folds. How do you choose? They are statistically tied; choose on practical grounds (handles missing values and interactions) or simplicity. Logistic regression would be defensible. We say so.

17. Why cross-validate by night? Rows from the same night share stars, noise and artifacts; random KFold would give inflated scores. GroupKFold keeps each night entirely in one fold.

18. Why is "bogus tracklets per night" more useful than AP here? 96% of our tracklets are real because we plant 300 fakes per night, so AP saturates (about 0.999). What a human experiences is how many junk tracks pass the threshold per night (about 2 from 9 validation examples: "a few").

19. The test night showed 9 false positives. What did you check? We compared them with all known asteroids: all 9 were real known objects (V 17.5-21.2) that our labels missed (fainter than the V <= 20.5 label cut, or a frame with a failed match). True false positives: 0; the labels were wrong.

C. Metrics

20. Why is accuracy useless here? "Always bogus" scores 96.8% with zero recall; the baseline scores 98.4% with 72.7% recall. Accuracy mostly measures the common class.

21. Define recall, precision and false-positive rate with our numbers. TP 1,547, FN 582, FP 493, TN 64,419. Recall = 1,547 / 2,129 = 72.7%; precision = 1,547 / 2,040 = 75.8%; FPR = 493 / 64,912 = 0.76%.

22. ROC AUC was 0.95 and AP 0.78. Why the difference? ROC divides false positives by all 65,000 negatives, so hundreds of false alarms look tiny; precision divides by the number of alerts. With rare positives, precision-recall is the honest picture.

23. How and why do you choose the threshold at a recall target? A missed asteroid is a lost discovery, a false alarm costs a minute, so fix recall (90%) on validation and minimise false alarms. For the baseline:

threshold 0.082, precision 19.3%, 804 false positives per frame. Never choose on test.

24. Worked example: 2,099 real, 64,974 bogus, FPR 2%, recall 95%. What is the precision? TP = 0.95 x 2,099 = 1,994; FP = 0.02 x 64,974 = 1,299; precision = 1,994 / (1,994 + 1,299) = 60.6%.

25. Worked example: a model trained on 50% real outputs 0.8, but real objects are 3%. What is the adjusted probability? Odds = 0.8 / 0.2 = 4. Prior-odds ratio = (0.03 / 0.97) / (0.5 / 0.5) = 0.0309. New odds = 4 x 0.0309 = 0.124; probability = 0.124 / 1.124 = 11%.

26. What is calibration and how do you do it? Making scores mean probabilities. Raw scores are inflated because training was balanced (16.7% real vs 3.1% true). Base-rate adjustment (Bayes in odds form) needs no fitting; isotonic regression fits a never-decreasing curve; fit on one half of validation and test on the other. Ranking is unchanged.

27. What does a completeness error bar mean? Standard error sqrt(p(1-p)/n): with p = 0.7 and n = 30 it is +/- 0.084, so differences between bins of 30 objects are mostly noise.

28. What is the baseline model and why have one? Logistic regression on six numbers per detection (AP 0.780). Every fancier model must beat it, or it is not worth the complexity.

D. Cross-match and vetting

29. How do you decide a tracklet is a known asteroid? At least half of its detections match the same catalogue object within 4 px (any brightness). Requiring several matches makes chance coincidences negligible.

30. A night gives 3 unknown candidates; validation predicts about 2 bogus per night. What do you conclude? Consistent with background: most are probably bogus; vet them, and treat any real one as needing a second night. A non-discovery is the expected outcome.

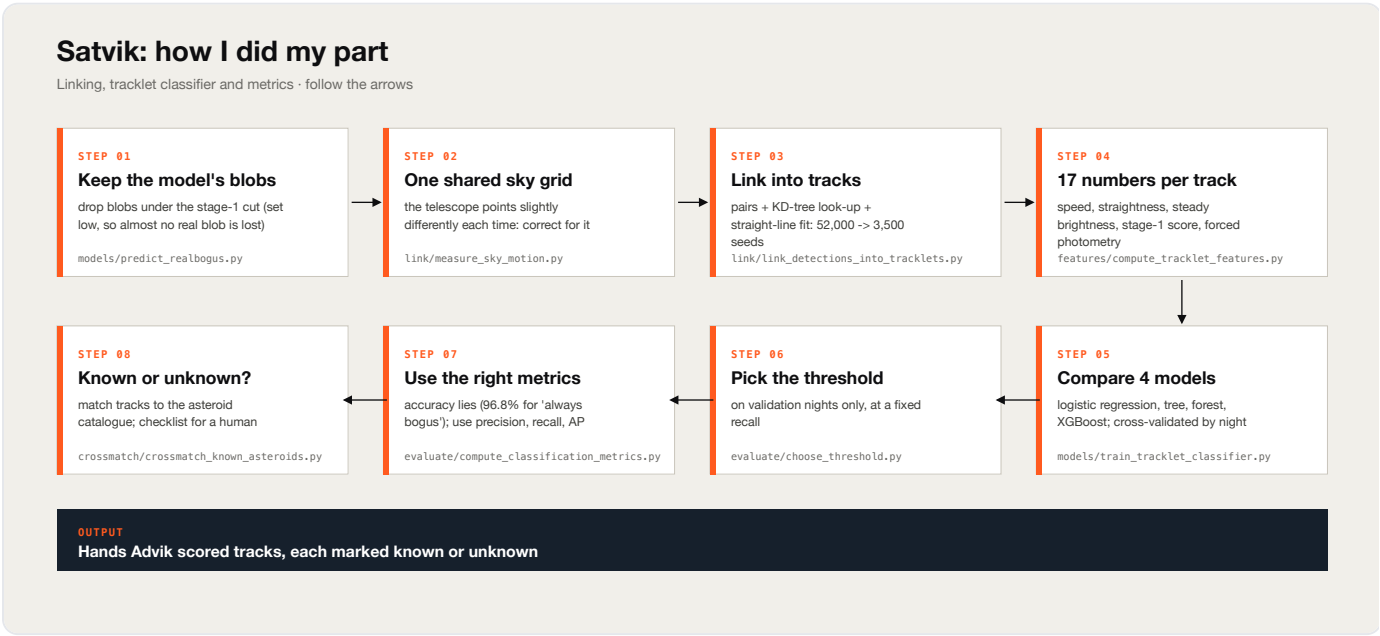
31. Walk through the vetting checklist. Stamps (science and difference show a point source, reference nothing, every frame); motion steady and plausible; brightness steady; not on a galaxy, bright star, bleed trail or satellite streak; no catalogue match at a looser radius; seen on another night.

32. Why look at the pictures if the numbers look convincing? On the test night track #45 had a steady main-belt-like motion and brightness, but the stamps showed it sat on an elongated galaxy visible in the reference: a subtraction residual. Numbers can mislead; pixels settle it.

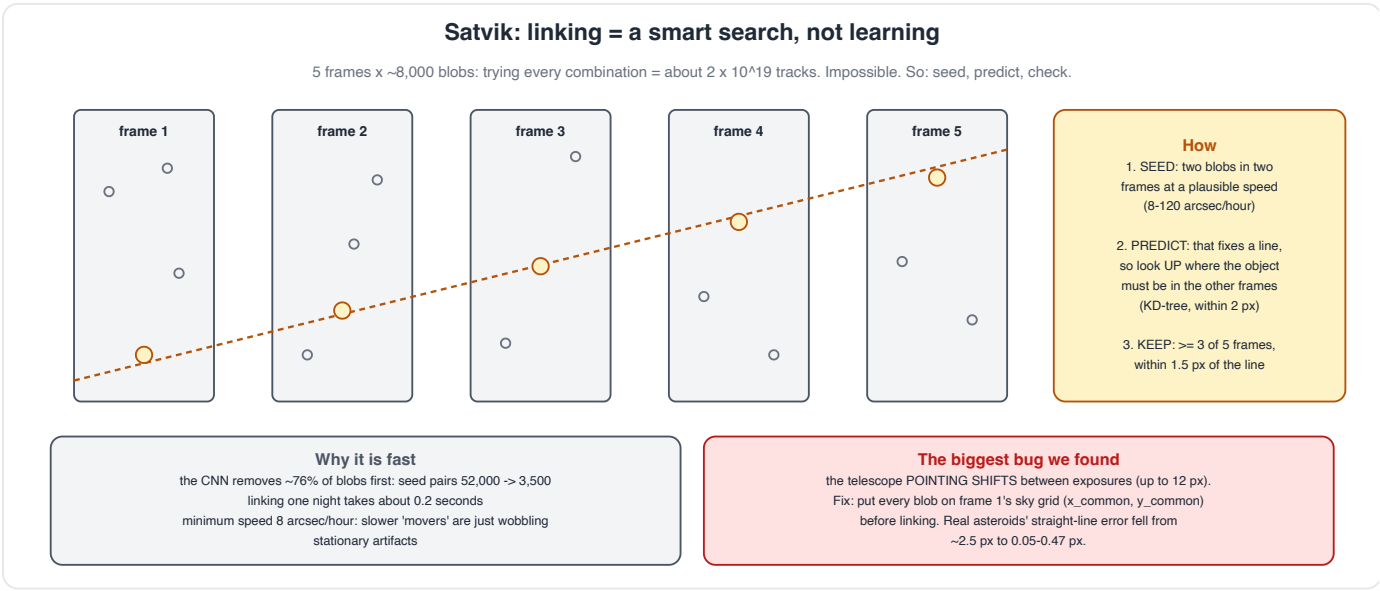
33. Why does the ADES draft use mid-exposure times? OBSJD is the shutter-open time; the object was at its recorded position on average 15 s later (JPL Horizons confirmed mid-exposure fits better: 0.13 vs 0.21 arcsec). The draft is marked "NOT FOR SUBMISSION": real reports need vetted multi-night objects.

34. "Did you write the models yourself?" Answer truthfully (see `00_Start_Here/07_AI_Use_and_Integrity.md`): the project was built with an AI coding assistant under Advik's direction; say what you re-ran or changed (for example the confusion matrix, the model comparison, the hand calculations) and explain a model without notes.

Diagrams



00 Process Diagram Satvik



01 linking

Satvik: from one decision tree to gradient boosting (XGBoost)

Decision tree
a single tree: a few yes/no questions, e.g. 'is the brightness steady?'

readable, but one tree over- or under-fits

→

Random forest (bagging)
hundreds of trees, each on a random sample of rows and features; AVERAGE their votes

lower variance

→

Gradient boosting
trees built ONE AFTER ANOTHER; each fits the ERRORS of the trees before it, scaled by a small learning rate

usually the best on tables

Our model (train_tracklet_classifier.py)
300 trees, depth 3, learning rate 0.05, each tree sees 80% of rows and 80% of columns. Settings are modest on purpose: only ~1,600 tracklets to learn from.

Honest result
cross-validated by NIGHT (GroupKFold): logistic regression 0.999, tree 0.983, forest 0.999, XGBoost 0.999 (average precision).
They TIE, so the choice is practical (handles missing values, interactions). A simpler model would also be defensible.

02 boosting

Satvik: choosing the operating point (fix recall first, then count the false alarms)

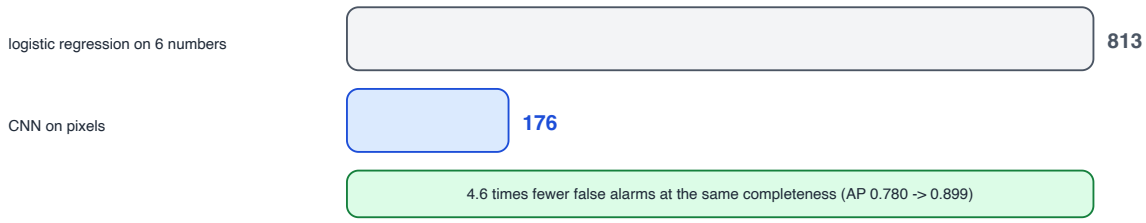
The cost argument
a missed asteroid = a lost discovery
a false alarm = one minute of a person's time

so: target 90% recall, then minimise false alarms

How we choose
threshold chosen on the VALIDATION nights (never on test):
'the cut that keeps 90% of real objects'

Do scores mean probabilities?
raw scores are inflated because training was balanced (16.7% real, truth 3.1%). Base-rate adjustment or isotonic calibration fixes the meaning, not the ranking

False alarms per frame at 90% recall (validation nights, 67,041 detections, 3.2% real)



4.6 times fewer false alarms at the same completeness (AP 0.780 -> 0.899)

03 operating point

Satvik: reading a confusion matrix (the logistic-regression baseline, validation nights)

	called real	called bogus
uly real	1,547 true positives	582 false negatives (missed)
y bogus	493 false positives (alarms)	64,419 true negatives

recall = 1,547 / 2,129 = 72.7%
(of all real objects, how many kept)

precision = 1,547 / 2,040 = 75.8%
(of everything flagged, how many real)

false-positive rate = 493 / 64,912 = 0.76%

accuracy = (1,547 + 64,419) / 67,041 = 98.4%

Why we do not report accuracy alone
Accuracy paradox: 'always say bogus' scores 96.8% accuracy and finds nothing (recall 0%). With rare positives, accuracy only measures the common class.
Use recall, precision and the precision-recall curve.

04 confusion metrics